

NEW NATIONAL ADVERTISING

ADMIN PANEL — PRODUCTION-READY FOCUS PROMPT

Purpose: This document defines exactly how the Admin Panel must work. The requirement is not merely that the Admin UI exists. Every change made from the Admin Panel must travel through the real production system, persist correctly, and appear on the real deployed website.

1. PRODUCTION ARCHITECTURE

- **Frontend:** Production website deployed on Render and accessed through the configured custom domain.
- **Backend/API:** Separate Render web service. The production frontend must communicate with this production API, not a Replit/local development API.
- **Database:** Firebase Firestore.
- **Authentication:** Firebase Authentication.
- **Images/files:** Firebase Storage.
- **Admin authorization:** authenticated Firebase user must be authorized as an admin. The expected Firestore user record is `users/{uid}` with role: "admin" and active: true.

2. THE CORE REQUIREMENT

Anything I add, edit, or delete from the Admin Panel must affect the real production website. Do not implement a UI-only mock, local-only state, localStorage-only solution, temporary in-memory data, or a disconnected Firebase collection that the frontend never reads.

The complete production flow must work end-to-end:

Admin Panel → Authentication → Admin authorization → API → Firestore/Storage → saved production data → production frontend → visible live website

3. PRODUCTS, MACHINES, SERVICES AND PROJECTS

The Admin Panel must provide real CRUD operations for the website content that is intended to be editable, including Products, Machines, Services, Projects, and Website Content.

- Add a new item with its required text fields.
- Upload an image from the Admin Panel when an image is required.
- Store the uploaded image in Firebase Storage and save the resulting usable image URL/reference with the Firestore document.
- Save the complete item to Firestore through the production data flow.
- Edit an existing item and persist the changes.
- Delete an item with a confirmation step and remove or safely handle its associated data.
- Support published/visibility and featured states wherever those controls already exist in the design.
- After refresh, the Admin Panel must load the same saved data from the production database.
- The public website must read the same production data and display the changes.

4. IMAGE UPLOAD REQUIREMENT

This is especially important. If I add a product, machine, project, service, or other item with an image, the image must not merely appear temporarily in the Admin Panel.

- The selected image must upload successfully to Firebase Storage.

- The upload must return a valid production-accessible URL/reference.
- That URL/reference must be stored with the correct Firestore record.
- The public website must use that stored image.
- Refreshing the Admin Panel must still show the saved image.
- Refreshing the public website must still show the saved image.
- Do not rely on a browser-local file path, object URL, temporary preview URL, or localStorage for permanent image storage.

5. ADMIN AUTHENTICATION & SECURITY

- Use the existing Firebase Authentication setup.
- Do not create a second unrelated authentication system.
- Only an authenticated authorized admin may access protected Admin Panel operations.
- Verify the user's admin status using the existing users/{uid} authorization model.
- Do not expose Firebase Admin SDK private credentials, service-account keys, or other secrets in frontend code.
- Production environment variables must be used for server-side secrets.

6. API / FIREBASE CONSISTENCY

Do not assume that because Firebase is connected, the Admin Panel is working. Test the actual CRUD operation.

- Verify the exact API request sent by the Admin Panel.
- Verify the backend receives the request and authenticates/authorizes it.
- Verify Firestore receives the intended document data.
- Verify Firebase Storage receives image uploads.
- Verify the backend response is handled correctly by the frontend.
- Verify the public frontend reads the same production record.
- Fix CORS, environment-variable, collection-path, field-name, authentication-token, permission, and API-route issues where they prevent the real flow from working.

7. DATA MODEL CONSISTENCY

The Admin Panel and public website must use the same agreed production data model. Do not create duplicate collections or alternate field names that cause the Admin Panel and frontend to read different data.

- Use stable document IDs.
- Keep field names consistent between create, read, update, and delete operations.
- Keep image URL/reference fields consistent.
- Do not silently discard fields during update operations.
- Do not overwrite existing records unintentionally when creating a new item.

8. USER EXPERIENCE

- Show a clear loading state while saving or uploading.
- Disable duplicate submissions while an operation is in progress.
- Show a clear success message after a confirmed production save.
- Show the actual error when an operation fails instead of silently doing nothing.

- Keep the Admin Panel simple and easy to use. Do not over-engineer it.
- Keep the Admin Panel responsive and usable on desktop and mobile.

9. MANDATORY END-TO-END TEST

Before saying the Admin Panel is fixed, perform a real production test. Do not test only by inspecting code.

Step	Required result
1. Log in as admin	Admin Panel opens and protected actions are authorized.
2. Add a new product	Form submits successfully.
3. Upload a product image	Image uploads to Firebase Storage.
4. Save product	Firestore record is created successfully.
5. Refresh Admin Panel	Product and image are still present.
6. Open live website	New product is visible using the saved production image.
7. Edit product	Changes persist after refresh.
8. Delete product	Product is removed from the production data source and no longer appears publicly.

10. PRODUCTION VERIFICATION

- Use the actual production frontend URL/domain, not only Replit preview.
- Use the actual production Render API, not localhost.
- Verify the frontend's production API URL points to the correct Render backend.
- Verify Render environment variables are configured correctly.
- Verify the production Firebase project is being used.
- After fixing the implementation, redeploy the affected Render service(s).
- Test the final deployed version from a clean browser refresh.

11. DO NOT CHANGE UNRELATED PARTS

Do not redesign the website, replace the existing visual design, remove working content, change unrelated routes, or migrate the project to another platform while fixing the Admin Panel. Preserve the existing Firebase data and authentication wherever possible.

12. FINAL ACCEPTANCE CRITERIA

The Admin Panel is considered complete only when a real admin can add a new item, upload its image, save it, refresh the Admin Panel, open the real deployed website, and see the same saved item and image there.

The same must work for editing and deleting. If any part of this chain fails, identify the failing layer, fix it, redeploy when necessary, and repeat the end-to-end test.

Important: Do not respond with "Firebase is connected," "the API exists," or "the feature should work." The required proof is an actual successful end-to-end production test.